K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

UNIT 1 ASSIGNMENT

AERT - Algorithmic Efficiency & Recursion Toolkit

Data Structures (ETCCDS202)

Name of the School:	School of Engineering & Technology
Program:	B.Tech CSE (AI and ML)
Course Title:	Data Structures
Course Code:	ETCCDS202
Unit Title:	Foundations & Algorithmic Analysis
Student Name:	GAURI SHARMA
Roll Number:	2501730108
Section:	A
Semester:	2
Batch:	2025-26
Submitted To:	Mrs. Neetu Chauhan
Submission Date:	5th March 2026

THIS PROJECT IS PART OF THE DATA STRUCTURES (UNIT 1) ASSIGNMENT.

ASSIGNMENT OVERVIEW:

• DATA STRUCTURES AND ADTS (WHAT THEY ARE, WHY WE USE THEM, AND COMMON OPERATIONS).

• ALGORITHMIC ANALYSIS (TIME/SPACE COMPLEXITY, BIG-O/BIG-Ω/BIG-Θ, BEST/AVERAGE/WORST CASES).

• ALGORITHM PARADIGMS (DIVIDE & CONQUER, GREEDY, DYNAMIC PROGRAMMING) – ONLY CONCEPTUALLY.

• RECURSION (BASE CASE, CALL STACK, AND HOW RECURSIVE SOLUTIONS WORK).

#MAIN TASK

BUILD: "ALGORITHMIC EFFICIENCY & RECURSION TOOLKIT (AERT)" IN PYTHON

MY TOOLKIT HAVES:

• ONE SMALL ADT IMPLEMENTATION (STACK ADT)

• RECURSIVE SOLUTIONS FOR FACTORIAL AND FIBONACCI

• A RECURSIVE SOLUTION FOR TOWER OF HANOI (WITH TRACE FOR N = 3)

• A RECURSIVE SOLUTION FOR BINARY SEARCH

• A SHORT ANALYSIS REPORT EXPLAINING COMPLEXITY AND PARADIGMS

Part A – Stack ADT

- **What is an ADT?**
  An Abstract Data Type (ADT) is a logical model of a data structure that defines what operations can be performed without specifying how they are implemented internally. It focuses on behaviour, not implementation.
- **Why stack operations are O(1)?**
  Stack operations like push, pop, and peek operate only on the top element. They do not require traversal of elements. Therefore, the time taken does not depend on the size of the stack, making them constant time O(1).

Part B – Factorial & Fibonacci

- **Factorial**

Time Complexity:
O(n) in worst, average, and best case.

Space Complexity:
O(n) due to recursion stack.

Big-Ω = O(n)
Big-Θ = O(n)

- **Fibonacci (Naive)**

Time Complexity:
$O(2^n)$

Reason:
Each call makes two recursive calls, creating an exponential tree.

Space Complexity:
O(n) (recursion depth)

Big-Ω = $O(2^n)$
Big-Θ = $O(2^n)$

Why it becomes slow?
Because it recomputes the same Fibonacci values multiple times.

- **Fibonacci (Memoized)**

Time Complexity:
O(n)

Space Complexity:
O(n) (memo storage + recursion stack)

Why faster?
Because each Fibonacci value is calculated only once and stored.

Paradigm:
Dynamic Programming (uses overlapping subproblems + memoization).

Part C – Tower of Hanoi

Number of moves = $2^n - 1$

Reason:
Each problem of size n reduces into two problems of size n−1.

Time Complexity:
$O(2^n)$

Space Complexity:
O(n) recursion depth.

Uses recursion with base case n = 1.

Part D – Recursive Binary Search

Recurrence relation:

$T(n) = T(n/2) + O(1)$

Time Complexity:

Worst case: $O(\log n)$
Average case: $O(\log n)$
Best case: $O(1)$ (element found at middle)

Space Complexity: $O(\log n)$

Paradigm: Divide & Conquer

Reason:
We divide the array into two halves and search only one half.

Part E – Paradigm Reflection

1. Binary Search: The "Divide & Conquer"

Binary Search is the textbook definition of **Divide & Conquer** because it follows a three-step process:

1. **Divide:** Break the sorted array into two halves by finding the middle element.
2. **Conquer:** Compare the target to the middle. If it's not a match, "conquer" the relevant half by repeating the process.
3. **Combine:** In this specific case, the "combine" step is trivial (or non-existent) because once the element is found, the solution is immediate.

Why it fits: It reduces a large problem into a single, smaller sub-problem of the same type until the search space is exhausted.

---

2. Memoized Fibonacci: The DP Solution

Memoization is the "memory" of **Dynamic Programming (DP)**.

What problem does it solve?

It solves the **Overlapping Subproblems** problem. In a naive recursive Fibonacci tree, the algorithm calculates Fib(3) or Fib(4) dozens of times. Memoization ensures that once Fib(n) is computed, it is stored in a table (like a cache).

- **The Result:** Instead of an exponential time complexity of $O(2^n)$, it drops to linear $O(n)$ because each number in the sequence is only actually calculated **once**.

---

3. Tower of Hanoi: Recursion in Action

The Tower of Hanoi is a perfect example of reducing a complex task into a simpler version of itself.

- **The Smaller Subproblem:** To move n disks from Post A to Post C, you must first move n-1 disks to a "buffer" post (Post B). Once those are out of the way, you move the largest disk. Then, you solve the problem again for the n-1 disks.
- **The Base Case:** The recursion stops when n = 1. Moving a single disk is a trivial operation that requires no further sub-division.

---

4. Real-Life Examples

Divide & Conquer: A Wardrobe Sort

Imagine you have a giant pile of laundry. You **divide** it into smaller piles (Shirts, Pants, Socks). You then sort each small pile individually (**conquer**). It is much faster than looking at the entire mountain of clothes and trying to pick out one specific item at a time.

Dynamic Programming: Google Maps/GPS

When you drive from Home to the Office, your GPS has already calculated the best route for various segments of the city. If you take a detour, the GPS doesn't recalculate the entire world map; it uses the **stored optimal paths** from your new location to the destination that it already knows.

Greedy: The Cashier's Method

If you owe someone **35/-** and want to give them the fewest number of bills possible, you act "greedily":

1. Take the largest bill possible that is **<= 37/- (a 20/-).**
2. The remaining is **17/-.** Take the largest possible **(10/-).**
3. The remaining is **7/-.** Take the largest possible **(5/-).**
4. Finally, **two 1/-** bills.

You make the best choice *at that moment* without looking ahead, hoping it leads to the overall best solution.